

vLLM: Business Rules and Semantics

How the serving engine behaves, written as plain rules

How to read this

These are the rules the vLLM serving engine (the V1 engine) follows, written in plain language. They come from the source code, not from design documents. Each rule names the file and symbol it comes from, so you can check it there.

- Rules are numbered by area. For example, **P3** is rule 3 under *Sampling parameters*.
 - A *request* is one prompt sent to the engine. With `n > 1` it fans out into child requests.
 - A *token budget* is the most tokens the scheduler may run in one engine step.
 - A *block* is a fixed-size page of KV cache (16 tokens by default). Prefix caching shares full blocks.
 - *Prefill* is computing the prompt. *Decode* is generating output one token at a time.
 - *Preemption* means taking a running request off the GPU and putting it back in the waiting queue.
 - File paths are relative to the `vllm/` package folder.
-

1. Request lifecycle (R)

R1. A request is either generative or pooling, never neither. It must carry `SamplingParams` or `PoolingParams`. A pooling request has `max_tokens = 1`.

Source: `v1/request.py` (`Request.__init__`).

R2. A new request starts as WAITING. If it uses structured output it starts as `WAITING_FOR_STRUCTURED_OUTPUT_GRAMMAR` instead, and only becomes `WAITING` once its grammar is ready.

Source: `v1/request.py` (`Request.__init__`); `v1/core/sched/scheduler.py` (`_try_promote_blocked_waiting_request`).

R3. Every status after PREEMPTED counts as finished. The finished statuses are `STOPPED`, `LENGTH_CAPPED`, `ABORTED`, `IGNORED`, `ERROR` and `REPETITION`.

Source: `v1/request.py` (`RequestStatus`, `RequestStatus.is_finished`).

R4. Each finished status maps to one of five finish reasons. `STOPPED` → `stop`, `LENGTH_CAPPED` → `length`, `ABORTED` → `abort`, `ERROR` → `error`, `REPETITION` → `repetition`. `IGNORED` (prompt too long) also reports `length`, as in the OpenAI API.

Source: `v1/request.py` (`_FINISHED_REASON_MAP`); `v1/engine/__init__.py` (`FinishReason`, `FINISH_REASON_STRINGS`).

R5. The engine makes request IDs unique. It keeps the caller's ID as `external_req_id` and adds 8 random characters to the internal ID. Callers may not set `external_req_id` themselves.

`VLLM_DISABLE_REQUEST_ID_RANDOMIZATION` turns this off (deprecated, and duplicates may then break).

Source: `v1/engine/input_processor.py` (`InputProcessor.assign_request_id`).

R6. Aborting an unknown or already-finished request does nothing. Abort only acts on live requests. Aborting a parent request also aborts its child requests.

Source: `v1/core/sched/scheduler.py` (`Scheduler.finish_requests`); `v1/engine/output_processor.py` (`OutputProcessor.abort_requests`).

R7. A pooling request finishes as soon as it has output. Its status becomes `FINISHED_STOPPED`.

Source: `v1/core/sched/scheduler.py` (`Scheduler.update_from_output`).

R8. An error finish reason becomes an HTTP 500. The OpenAI server raises `GenerationError("Internal server error")` with error type `InternalServerError`.

Source: `entrypoints/openai/engine/serving.py` (`_raise_if_error, _convert_generation_error_to_streaming_response`); `v1/engine/__init__.py` (`FinishReason` docstring).

R9. Parallel sampling ($n > 1$) runs as n child requests. Each child has $n = 1$. If a seed is set, child i uses `seed + i`, so the children differ but stay reproducible.

Source: `v1/engine/parallel_sampling.py` (`ParentRequest._get_child_sampling_params`).

2. Input validation (I)

I1. The model must support the task. Sampling params on a model with no generation task fail with “This model does not support generation”. Pooling params on a model with no pooling task fail the same way.

Source: `v1/engine/input_processor.py` (`InputProcessor._validate_params`).

I2. A decoder prompt cannot be empty.

Source: `v1/engine/input_processor.py` (`InputProcessor._validate_prompt_len`).

I3. A prompt longer than `max_model_len` is rejected. For a generative model a prompt exactly equal to `max_model_len` is also rejected, because there is no room for even one output token.

Source: `v1/engine/input_processor.py` (`InputProcessor._validate_prompt_len`).

I4. Prompt token IDs must be in the vocabulary. The limit is the larger of the tokenizer's `max_token_id` and the model's vocab size minus 1.

Source: `v1/engine/input_processor.py` (`InputProcessor._validate_model_input`).

I5. One multimodal item must fit in the encoder cache. An image (or other item) with more embedding tokens than the encoder cache size is rejected.

Source: `v1/engine/input_processor.py` (`InputProcessor._validate_model_input`).

I6. Unset `max_tokens` means “fill the context”. The engine sets it to `max_model_len` minus the prompt length.

Source: `v1/engine/input_processor.py` (`InputProcessor.process_inputs`).

I7. In the API server, `max_tokens` is the smallest of several caps. It takes the minimum of: room left in the context, the request value (or the model's default), any server override, and the platform's limit.

Source: `entrypoints/utils.py` (`get_max_tokens`).

I8. A LoRA request needs LoRA to be enabled. Otherwise it fails with “LoRA is not enabled”.

Source: `v1/engine/input_processor.py` (`InputProcessor._validate_lora`).

I9. `thinking_token_budget` needs a reasoning config. Setting it without `--reasoning-config` is an error.

Source: `v1/engine/input_processor.py` (`InputProcessor._validate_params`).

I10. A requested data-parallel rank must exist. It must be in `[0, number of ranks)`.

Source: `v1/engine/input_processor.py` (`InputProcessor.process_inputs`).

3. Sampling parameters (P)

P1. n must be an integer from 1 to `VLLM_MAX_N_SEQUENCES` (default 16,384).

Source: `sampling_params.py` (`SamplingParams._verify_args`); `envs.py` (`VLLM_MAX_N_SEQUENCES`).

P2. Penalties have fixed ranges. `presence_penalty` and `frequency_penalty` must be in `[-2, 2]`. `repetition_penalty` must be greater than 0 (1.0 means off).

Source: `sampling_params.py` (`SamplingParams._verify_args`).

P3. Temperature 0 means greedy. Below `1e-5` the request is greedy: `top_p`, `top_k` and `min_p` are reset to off, and n must be 1. Negative temperature is rejected.

Source: `sampling_params.py` (`SamplingParams.__post_init__`, `_verify_greedy_sampling`, `sampling_type`).

P4. Very small temperatures are raised to 0.01. A value between 0 and 0.01 is bumped to 0.01 with a warning, to avoid NaN or inf in the maths.

Source: `sampling_params.py` (`SamplingParams.__post_init__`, `_MAX_TEMP`).

P5. `top_p` must be in (0, 1]. `min_p` must be in [0, 1]. `top_k` must be 0 (off) or at least 1. `top_k = -1` is quietly accepted as “off”. `top_k` must be an integer.

Source: `sampling_params.py` (`SamplingParams._verify_args`).

P6. `max_tokens` must be at least 1. `min_tokens` must be 0 or more and not above `max_tokens`.

Source: `sampling_params.py` (`SamplingParams._verify_args`).

P7. A seed of -1 means no seed. A request with a seed is sampled with its own generator (`RANDOM_SEED`); without one it is plain `RANDOM`.

Source: `sampling_params.py` (`SamplingParams.__post_init__`, `sampling_type`).

P8. Logprob counts are capped by `max_logprobs` (default 20). `logprobs` and `prompt_logprobs` may be -1, meaning the whole vocabulary, but that still has to fit under the cap. A cap of -1 means no cap.

Source: `sampling_params.py` (`SamplingParams._validate_logprobs`); `config/model.py` (`ModelConfig.max_logprobs`).

P9. `logprob_token_ids` can list at most 128 tokens. If `logprobs` is also set, it must equal the list length.

Source: `sampling_params.py` (`MAX_LOGPROB_TOKEN_IDS`, `SamplingParams._validate_logprobs`).

P10. Token-level controls must use real token IDs. `logit_bias` keys and `allowed_token_ids` must be inside the vocabulary. `allowed_token_ids` cannot be an empty list. Every token of every `bad_words` entry must also be in range.

Source: `sampling_params.py` (`_validate_logit_bias`, `_validate_allowed_token_ids`, `update_from_tokenizer`).

P11. Bad words are blocked both at the start and in the middle of text. Each word is tokenized with and without a leading space, so both forms are banned.

Source: `sampling_params.py` (`SamplingParams.update_from_tokenizer`).

P12. Speculative decoding does not support `min_p` or `logit_bias`. A request with either is rejected when a speculative config is active.

Source: `sampling_params.py` (`SamplingParams._validate_spec_decode`).

P13. Asking for prompt logprobs turns off prefix-cache reads for that request. Cached tokens would not have logprobs, so the request recomputes its whole prompt.

Source: `sampling_params.py` (`SamplingParams.__post_init__`); `v1/core/kv_cache_manager.py` (`KVCacheManager.get_computed_blocks`).

4. Stopping and finish reasons (S)

S1. Nothing stops a request before `min_tokens`. The `min_tokens` check runs first, so EOS, stop tokens, the length cap and repetition detection are all ignored until it is met.

Source: `v1/core/sched/utils.py` (`check_stop`).

S2. After that, stop checks run in a fixed order. EOS token → stop token IDs → length (`max_model_len` or `max_tokens` reached) → repetition. The first match decides the finish reason.

Source: `v1/core/sched/utils.py` (`check_stop`).

S3. `ignore_eos` only ignores EOS. The EOS token is not set as a stop condition, but `stop_token_ids` and stop strings still apply.

Source: `sampling_params.py` (`SamplingParams.update_from_generation_config`); `v1/core/sched/utils.py` (`check_stop`).

S4. Extra EOS IDs from the model's `generation_config` act as stop tokens. Unless `ignore_eos` is set, they are added to `stop_token_ids`.

Source: `sampling_params.py` (`SamplingParams.update_from_generation_config`).

S5. A stop token reports itself as the stop reason. The stop token ID is stored in `stop_reason`. For repetition the reason is the string `"repetition_detected"`.

Source: `v1/core/sched/utils.py` (`check_stop`).

S6. Stop strings need detokenization. Using `stop` with `detokenize=False` is rejected. An empty string in `stop` is rejected.

Source: `sampling_params.py` (`SamplingParams._verify_args`).

S7. Stop strings are cut out of the output by default. The text is truncated at the start of the match. With `include_stop_str_in_output=True` it is truncated at the end of the match instead.

Source: `v1/engine/detokenizer.py` (`check_stop_strings`).

S8. Streaming holds back a few characters. While a request runs, the last `longest stop string - 1` characters are not streamed yet, so a partial stop string is never sent. They are released at the end.

Source: `v1/engine/detokenizer.py` (`BaseIncrementalDetokenizer.__init__`, `get_next_output_text`).

S9. Stop strings are found in the front end, then the engine is told to abort. If the detokenizer sees a stop string before the engine has finished the request, the request is aborted in the engine core.

Source: `v1/engine/output_processor.py` (`OutputProcessor.process_outputs`, `reqs_to_abort`).

S10. Repetition detection looks for a repeating N-gram at the end of output. It is off by default (`max_pattern_size = 0`). When on, `min_count` must be at least 2 and `min_pattern_size` (0 means 1) must not exceed `max_pattern_size`.

Source: `sampling_params.py` (`RepetitionDetectionParams.__post_init__`); `v1/core/sched/utils.py` (`check_sequence_repetition`).

S11. Output tokens after a stop are dropped. When several tokens arrive in one step (for example with speculative decoding), tokens after the stopping one are trimmed.

Source: `v1/core/sched/scheduler.py` (`Scheduler.update_request_with_output`).

5. Scheduling, batching and preemption (C)

C1. There is no separate prefill phase or decode phase. Each step, the scheduler gives each request enough tokens to catch its computed count up to its total count. This one rule covers chunked prefill, prefix caching and speculative decoding.

Source: `v1/core/sched/scheduler.py` (`Scheduler.schedule`, opening note).

C2. Running requests are scheduled before waiting ones. New or resumed requests only get the token budget left over.

Source: `v1/core/sched/scheduler.py` (`Scheduler.schedule`).

C3. One step has a token budget and a request limit. The budget is `max_num_scheduled_tokens` (defaults to `max_num_batched_tokens`). No new request is admitted once `max_num_seqs` requests are running.

Source: `v1/core/sched/scheduler.py` (`Scheduler.__init__`, `Scheduler.schedule`).

C4. Long prompts are cut into chunks. With chunked prefill, a request takes at most the remaining budget, and at most `long_prefill_token_threshold` tokens when that is set. Without chunked prefill, a waiting request that does not fit the budget stops admission for that step.

Source: `v1/core/sched/scheduler.py` (`Scheduler.schedule`); `config/scheduler.py` (`SchedulerConfig`).

C5. A request never runs past `max_model_len`. Tokens scheduled in one step are capped so the position stays below `max_model_len`.

Source: `v1/core/sched/scheduler.py` (`Scheduler.schedule`).

C6. When KV cache runs out, the scheduler preempts. Under `fcfs` it preempts the last request in the running list. Under `priority` it preempts the request with the highest priority value, latest arrival breaking ties. If the victim is the request being scheduled, it stops trying.

Source: `v1/core/sched/scheduler.py` (`Scheduler.schedule`).

C7. Preemption throws away the KV cache. The request's blocks are freed, its computed count goes back to 0, draft tokens are dropped, and it goes to the front of the waiting queue. It is recomputed later (possibly helped by prefix-cache hits).

Source: `v1/core/sched/scheduler.py` (`Scheduler._preempt_request`).

C8. A step that preempted anything admits no new requests.

Source: `v1/core/sched/scheduler.py` (`Scheduler.schedule, if not preempted_reqs`).

C9. Priority order is: lower priority value first, then earlier arrival, then request ID. This order is only used by the `priority` policy. Under `fcfs` (the default) requests are served in arrival order and the `priority` field is not used.

Source: `v1/request.py` (`Request.__lt__`); `v1/core/sched/request_queue.py` (`FCFSRequestQueue, PriorityRequestQueue`).

C10. Blocked requests are skipped, not waited on. Requests waiting for a grammar, for remote KV, or for the next streaming input are set aside so others can run.

Source: `v1/core/sched/scheduler.py` (`_is_blocked_waiting_status, Scheduler.schedule`).

C11. At most `max_loras` different adapters run in one step. A waiting request that would add one more adapter is skipped for that step.

Source: `v1/core/sched/scheduler.py` (`Scheduler.schedule, scheduled_loras`).

C12. A paused engine schedules nothing. In the `PAUSED_ALL` state the token budget is 0.

Source: `v1/core/sched/scheduler.py` (`Scheduler.schedule, PauseState.PAUSED_ALL`).

C13. By default a new request is admitted only if its whole prompt fits. With `scheduler_reserve_full_isl = True` the scheduler checks KV space for the full input, not just the first chunk, to avoid admitting too much and then thrashing.

Source: `config/scheduler.py` (`SchedulerConfig.scheduler_reserve_full_isl`); `v1/core/kv_cache_manager.py` (`KVCacheManager.allocate_slots`).

6. KV cache and prefix caching (K)

K1. Only full blocks are cached. Block hashes are computed only for complete blocks of `block_size` tokens. A partly filled last block is never shared.

Source: `v1/core/kv_cache_utils.py` (`get_request_block_hasher`).

K2. A block's hash covers everything before it. The hash combines the parent block's hash, the block's tokens, and extra keys. So two blocks match only if their whole prefix matches.

Source: `v1/core/kv_cache_utils.py` (`hash_block_tokens`).

K3. Extra keys keep caches apart. They include the LoRA name, multimodal input hashes, prompt-embedding hashes, and the `cache_salt`. The salt is only added to the first block, which is enough to separate the whole chain.

Source: `v1/core/kv_cache_utils.py` (`generate_block_hash_extra_keys, _gen_lora_extra_hash_keys`).

K4. The last prompt token is always recomputed. Even on a full cache hit, the hit is capped at `num_tokens - 1` so the model can produce logits. This can recompute a whole block.

Source: `v1/core/kv_cache_manager.py` (`KVCacheManager.get_computed_blocks`).

K5. Some requests skip reading the cache. Prefix caching is skipped when it is disabled, or when the request sets `skip_reading_prefix_cache` (prompt logprobs, and `token_embed/token_classify` pooling by default).

Source: `v1/core/kv_cache_manager.py` (`get_computed_blocks`); `pooling_params.py` (`_merge_default_parameters`).

K6. Freed blocks stay cached until reused (LRU). A freed block goes to the back of the free queue with its hash intact. A later request can still hit it. It is only evicted when it is handed out again.

Source: `v1/core/block_pool.py` (`BlockPool.free_blocks`, `get_new_blocks`, `_maybe_evict_cached_block`); `v1/core/kv_cache_utils.py` (`FreeKVCacheBlockQueue`).

K7. A request's tail blocks are evicted first. Blocks are freed in reverse order, so the end of a chain (least likely to be shared) is reused before the shared start.

Source: `v1/core/single_type_kv_cache_manager.py` (`SingleTypeKVCacheManager.free`).

K8. A cache hit takes a block back out of the free queue. “Touching” a block with ref count 0 removes it from the free list and raises its ref count.

Source: `v1/core/block_pool.py` (`BlockPool.touch`).

K9. Resetting the prefix cache only works when no block is in use. Otherwise it fails with a warning. The scheduler can force it by preempting every running request first.

Source: `v1/core/block_pool.py` (`BlockPool.reset_prefix_cache`); `v1/core/sched/scheduler.py` (`Scheduler.reset_prefix_cache`).

K10. Hashes differ per process unless PYTHONHASHSEED is set. The root hash is random by default. With a CBOR hash function and no seed, vLLM warns that hashes are not reproducible.

Source: `v1/core/kv_cache_utils.py` (`init_none_hash`).

K11. A failed remote KV load fails the request by default. `kv_load_failure_policy` is `fail` (finish with error). Set it to `recompute` to recompute the missing blocks instead.

Source: `config/kv_transfer.py` (`KVTransferConfig.kv_load_failure_policy`); `v1/core/sched/scheduler.py` (`Scheduler.__init__`).

7. Structured output (G)

G1. A request uses exactly one kind of constraint. One of `json`, `regex`, `choice`, `grammar`, `json_object` or `structural_tag`. None or more than one is an error.

Source: `sampling_params.py` (`StructuredOutputsParams.__post_init__`).

G2. choice cannot be an empty list. grammar cannot be empty or only whitespace.

Source: `sampling_params.py` (`SamplingParams._validate_structured_outputs`).

G3. Structured output needs a tokenizer. It cannot be used with `skip_tokenizer_init`.

Source: `sampling_params.py` (`SamplingParams._validate_structured_outputs`).

G4. The backend is chosen per engine, not per request. A request that names a different backend than the engine's is rejected. Only one backend is created per engine.

Source: `sampling_params.py` (`_validate_structured_outputs`); `v1/structured_output/__init__.py` (`StructuredOutputManager.grammar_init`).

G5. auto tries xgrammar, then guidance, then outlines. If xgrammar cannot handle the request, guidance is used. Outlines is used instead when the tokenizer is a non-tekken Mistral one or the JSON schema has features guidance does not support.

Source: `sampling_params.py` (`SamplingParams._validate_structured_outputs`).

G6. If the grammar rejects a generated token, the request ends with error.

Source: `v1/core/sched/scheduler.py` (`Scheduler.update_from_output`).

G7. With a reasoning parser, the grammar starts after reasoning ends. The model reasons freely, then the constraint applies. `enable_in_reasoning=True` constrains the reasoning too.

Source: `v1/structured_output/__init__.py` (`should_fill_bitmask`, `should_advance`).

G8. Some options only work with some backends. `disable_any_whitespace` needs xgrammar or guidance. `disable_additional_properties` needs guidance.

Source: `config/structured_outputs.py` (`StructuredOutputsConfig._validate_structured_output_config`).

8. LoRA adapters (L)

L1. A LoRA ID must be 1 or more, and the path cannot be empty. The ID must be unique per adapter.

Source: `lora/request.py` (`LoRARequest.__post_init__`).

L2. An adapter's rank cannot exceed `max_lora_rank` (default 16). DoRA, adapter bias and `modules_to_save` are not supported.

Source: `lora/peft_helper.py` (`PEFTHelper.validate_legal`, `_validate_features`); `config/lora.py` (`LoRAConfig`).

L3. `max_cpu_loras` must be at least `max_loras`. If unset it equals `max_loras`.

Source: `config/lora.py` (`LoRAConfig._validate_lora_config`).

L4. Adapters are kept with LRU eviction. When the GPU slots (`max_loras`) or CPU cache (`max_cpu_loras`) are full, the least recently used adapter is removed. Pinned adapters are kept.

Source: `lora/model_manager.py` (`LRUCacheLoRAModelManager.activate_adapter`, `add_adapter`, `pin_adapter`).

L5. Different adapters never share prefix cache. The LoRA name is part of every block hash (see K3).

Source: `v1/core/kv_cache_utils.py` (`_gen_lora_extra_hash_keys`).

L6. Loading adapters at runtime is off by default. The load/unload endpoints only exist when `VLLM_ALLOW_RUNTIME_LORA_UPDATING` is set, which logs a "local development only" warning. It cannot be used with more than one API server process.

Source: `entrypoints/serve/lora/api_router.py` (`attach_router`); `entrypoints/cli/serve.py` (`run_multi_api_server`).

L7. A runtime load needs a name and a path, and the name must be new. Re-using a loaded name is refused unless `load_inplace=True`. Unloading an unknown name is a `NotFound` error.

Source: `entrypoints/openai/models/serving.py` (`_check_load_lora_adapter_request`, `_check_unload_lora_adapter_request`).

9. OpenAI-compatible API (A)

A1. `stream_options` only work with `stream=True`.

Source: `entrypoints/openai/chat_completion/protocol.py` and `completion/protocol.py` (`validate_stream_options`).

A2. Prompt logprobs are not available when streaming. `prompt_logprobs` must be -1 or 0 or more.

Source: `entrypoints/openai/chat_completion/protocol.py` (`check_logprobs`).

A3. In chat, `top_logprobs` needs `logprobs=true`.

Source: `entrypoints/openai/chat_completion/protocol.py` (`check_logprobs`).

A4. `response_format` of type `json_schema` must include `json_schema`.

Source: `entrypoints/openai/chat_completion/protocol.py` (`validate_response_format`).

A5. Tool rules follow OpenAI. An empty `tools` array is rejected. If tools are given without `tool_choice`, it defaults to `auto`. Any `tool_choice` other than `none` needs `tools`. A named tool must match one of the given tools.

Source: `entrypoints/openai/chat_completion/protocol.py` (`check_tool_usage`).

A6. `continue_final_message` and `add_generation_prompt` cannot both be true.

Source: `entrypoints/openai/chat_completion/protocol.py` (`check_generation_prompt`).

A7. `cache_salt`, if given, must be a non-empty string.

Source: `entrypoints/openai/chat_completion/protocol.py` (`check_cache_salt_support`).

A8. A completion needs a prompt or prompt embeddings.

Source: `entrypoints/openai/completion/protocol.py` (`validate_prompt_and_prompt_embeds`).

A9. An unknown model name is an error, unless a LoRA can be resolved for it. The name may be the base model, a loaded LoRA, or (with runtime LoRA updates on) a LoRA resolved on demand.

Source: `entrypoints/openai/engine/serving.py` (`_check_model`).

10. Pooling (O)

O1. A pooling request with no task gets one. The order tried is `token_embed`, then `token_classify`, then `plugin`. An unsupported task is rejected.

Source: `v1/engine/input_processor.py` (`InputProcessor._validate_params`).

O2. Each task only accepts its own parameters. Passing a parameter that the task does not use is an error.

Source: `pooling_params.py` (`PoolingParams._verify_valid_parameters`).

O3. Changing embedding size needs a Matryoshka model. `dimensions` is rejected on other models, and must be in the model's list of dimensions if it has one.

Source: `pooling_params.py` (`PoolingParams._set_default_parameters`).

Configuration constraints and defaults

Scheduler (`config/scheduler.py`, `SchedulerConfig`):

Setting	Default	Rule
<code>policy</code>	<code>fcfs</code>	<code>fcfs</code> or priority (C6, C9)
<code>max_num_batched_tokens</code>	2,048 (class default)	≥ 1 ; must be $\geq$ <code>max_num_seqs</code>
<code>max_num_seqs</code>	128 (class default)	≥ 1
<code>enable_chunked_prefill</code>	<code>true</code>	if false, <code>max_num_batched_tokens</code> must be $\geq$ <code>max_model_len</code>
<code>max_num_partial_prefills</code>	1	> 1 needs chunked prefill
<code>max_long_partial_prefills</code>	1	must be $\leq$ <code>max_num_partial_prefills</code>
<code>long_prefill_token_threshold</code>	0 (off)	becomes 4% of <code>max_model_len</code> when partial prefills > 1 ; must be $\leq$ <code>max_model_len</code>
<code>scheduler_reserve_full_isl</code>	<code>true</code>	see C13
<code>stream_interval</code>	1	≥ 1 token
encoder-decoder models	—	chunked prefill forced off

Real defaults set by `EngineArgs` (`engine/arg_utils.py`, `EngineArgs.get_batch_defaults`):

Hardware	<code>max_num_batched_tokens</code> (LLM / API server)	<code>max_num_seqs</code> (LLM / API server)
GPU $\geq$ 70 GiB, not A100	16,384 / 8,192	1,024 / 1,024
Other GPUs	8,192 / 2,048	256 / 256
CPU ($\times$ world size)	4,096 / 2,048	256 / 128

KV cache (`config/cache.py`, `CacheConfig`):

Setting	Default	Rule
<code>block_size</code>	16	
<code>gpu_memory_utilization</code>	0.92	> 0 and ≤ 1 ; per instance
<code>kv_cache_memory_bytes</code>	unset	when set, <code>gpu_memory_utilization</code> is ignored
<code>enable_prefix_caching</code>	true	
<code>prefix_caching_hash_algo</code>	sha256	also <code>sha256_cbor</code> , <code>xxhash</code> , <code>xxhash_cbor</code>
<code>cache_dtype</code>	auto	auto = model dtype
<code>mamba_block_size</code>	unset	must be > 0 when set

Model and sampling (`config/model.py`, `ModelConfig`; `envs.py`):

Setting	Default	Rule
<code>max_model_len</code>	from model config	≥ -1 ; -1 or <code>auto</code> = largest length that fits in GPU memory; above the model's limit needs <code>VLLM_ALLOW_LONG_MAX_MODEL_LEN=1</code>
fallback when the model config gives no length	2,048	used by <code>_get_and_verify_max_len</code>
<code>max_logprobs</code>	20	-1 = no cap
<code>logprobs_mode</code>	<code>raw_logprobs</code>	<code>raw</code> = before logits processors
<code>seed</code>	0	
<code>VLLM_MAX_N_SEQUENCES</code>	16,384	upper bound for <code>n</code>

SamplingParams defaults (`sampling_params.py`): `n` 1, `temperature` 1.0, `top_p` 1.0, `top_k` 0, `min_p` 0.0, `presence_penalty` / `frequency_penalty` 0.0, `repetition_penalty` 1.0, `max_tokens` 16 (class default; see I6 and I7), `min_tokens` 0, `skip_special_tokens` true, `output_kind` CUMULATIVE.

LoRA (`config/lora.py`, `LoRAConfig`):

Setting	Default	Rule
<code>max_lora_rank</code>	16	one of 1, 8, 16, 32, 64, 128, 256, 320, 512
<code>max_loras</code>	1	≥ 1 ; adapters per batch
<code>max_cpu_loras</code>	<code>= max_loras</code>	must be $\geq$ <code>max_loras</code>
<code>lora_dtype</code>	auto	auto = base model dtype

Structured output (`config/structured_outputs.py`): `backend` `auto` (also `xgrammar`, `guidance`, `outlines`, `lm-format-enforcer`); `enable_in_reasoning` false.

Parallelism (`config/parallel.py`, `ParallelConfig`): `tensor_parallel_size`, `pipeline_parallel_size` and `data_parallel_size` all default to 1. `enable_expert_parallel` defaults to false.

KV transfer (`config/kv_transfer.py`): `kv_load_failure_policy` `fail` (or `recompute`).